

Comparing Centralized and Local Differential Privacy Mechanisms Against Privacy Attacks

Project report for *Privacy-Preserving Methods for Data Science and Distributed Systems*

Department of Mathematics and Computer Science
University of Basel

Valdrin Sheremetaj, Ashraf Jafarli, Damaris Wahu
valdrin.sheremetaj@stud.unibas.ch, ashraf.jafarli@stud.unibas.ch,
damariswahu.mungai@stud.unibas.ch
2021-059-779, 2025-071-226, 2025-071-762

Abstract

Machine learning models trained on sensitive health data can provide useful predictions, but they may also leak information about the individuals used during training. In this project, we study this privacy risk on the CDC Diabetes Health Indicators dataset. We compare centralized and local privacy mechanisms for a binary diabetes classification task and evaluate both model utility and resistance against membership inference attacks. As models, we use Logistic Regression, a standard Multi-Layer Perceptron, and an intentionally overfitting MLP. As privacy mechanisms, we test a non-private baseline, Laplace parameter noise, Gaussian noise for Logistic Regression, and local differential privacy using randomized response on the input features. We evaluate utility with accuracy, F1-score, and AUC, and privacy leakage with a threshold-based membership inference attack and a shadow-model attack. Our results show that Logistic Regression and the standard MLP leak little membership information, with attack AUC values close to random guessing. In contrast, the overfitting MLP shows clear membership leakage, especially under the shadow-model attack. Strong privacy settings can reduce this leakage, but they also reduce model utility. Overall, the experiments confirm the privacy–utility trade-off and show that privacy mechanisms are most useful when the underlying model actually leaks membership information.

June 18, 2026

1 Introduction

Machine learning is often used on sensitive data, especially in areas such as health care. This is useful because models can find patterns in large datasets and support predictions, for example whether a person has diabetes or not. At the same time, this also creates privacy risks. A trained model does not store the training data directly, but it can still leak information about the data it was trained on. This is especially problematic for medical data, because even knowing that a person was part of a diabetes dataset can already reveal private information about their health.

In this project, we study this problem using the CDC Diabetes Health Indicators dataset. The task is formulated as a binary classification problem, where the model predicts whether a person has diabetes based on health and lifestyle features such as BMI, blood pressure, physical activity, age, and similar indicators. The models output prediction probabilities, and these confidence scores are important because they can be used by an attacker. If a model is more confident on samples it has seen during training than on unseen samples, this difference can be exploited by a membership inference attack.

The goal of our project is therefore to compare different privacy mechanisms and analyze how well they protect against such attacks. We use a non-private baseline and compare it with noise-based privacy mechanisms, namely Laplace parameter noise, Gaussian noise for logistic regression, and local differential privacy using randomized response on the input data. These mechanisms are tested with different privacy budgets ϵ , where smaller values mean stronger privacy but usually also more noise and lower model performance.

We evaluate both sides of the trade-off. On the one hand, the model should still be useful, so we measure accuracy, F1-score, and AUC. On the other hand, the model should not reveal too much about its training data, so we evaluate two membership inference attacks: a simple threshold-based attack and a stronger shadow-model attack. In addition to logistic regression, we also test neural network models, including an intentionally overfitting MLP, because overfitting is one of the main reasons why membership inference attacks can become successful.

Overall, the main research question of this report is how centralized and local privacy mechanisms affect both model utility and privacy leakage on a realistic health dataset. We are especially interested in whether stronger privacy settings reduce the success of membership inference attacks, and how much predictive performance is lost in return.

2 Background

This section gives the basic background for the concepts we use in this project. Since our project is about privacy risks in machine learning, the most important topics are differential privacy, local differential privacy, and membership inference attacks. These topics are connected quite directly. Differential privacy tries to make sure that a single person does not have too much influence on the output of a computation. Membership inference attacks try to do the opposite: they try to find out whether a specific person was part of the training data.

2.1 Differential Privacy

Differential privacy is a formal way to describe privacy for algorithms that work on sensitive data. The basic idea is that the output of an algorithm should not change too much if one single record is added or removed from the dataset. In other words, an attacker should not be able to confidently tell whether one specific person was included in the data just by looking at the output. Differential privacy was introduced by Dwork et al. (2006) and is now one of the standard definitions used in privacy-preserving data analysis.

Definition 1 ((ϵ, δ) -Differential Privacy). *A randomized mechanism $\mathcal{M}$ satisfies (ϵ, δ) -differential privacy if for all neighboring datasets D and D' that differ in one individual record, and for all possible output sets S , the following holds:*

$$\Pr[\mathcal{M}(D) \in S] \leq e^\epsilon \Pr[\mathcal{M}(D') \in S] + \delta$$

The parameter ϵ controls how strong the privacy guarantee is. A smaller ϵ means stronger privacy, because the outputs for neighboring datasets must be very similar. However, this usually also means that more noise has to be added, which can make the model worse. A larger ϵ gives weaker privacy, but usually keeps more of the original model performance. The parameter δ allows a very small probability that the privacy guarantee does not hold. This is often used together with the Gaussian mechanism.

In practice, differential privacy is often achieved by adding random noise. How much noise is needed depends on the sensitivity of the function. The sensitivity describes how much the output of a function can change if one record in the dataset changes. For a function f , the global sensitivity can be written as:

$$\Delta f = \max_{D, D'} |f(D) - f(D')|$$

If one record can have a large influence on the output, then more noise is needed to hide this influence.

2.2 Laplace and Gaussian Mechanisms

Two common mechanisms for differential privacy are the Laplace mechanism and the Gaussian mechanism. The Laplace mechanism adds noise sampled from a Laplace distribution. For a function f , it can be written as:

$$\mathcal{M}(D) = f(D) + \text{Lap}\left(\frac{\Delta f}{\epsilon}\right)$$

This means that the amount of noise depends on the sensitivity Δf and on the privacy budget ϵ . If ϵ is small, then the noise becomes larger. This is also the behaviour we expect in our experiments: stronger privacy should usually reduce the model performance, because the model receives more noise.

The Gaussian mechanism works in a similar way, but it adds normally distributed noise instead of Laplace noise. It is usually used for (ϵ, δ) -differential privacy:

$$\mathcal{M}(D) = f(D) + \mathcal{N}(0, \sigma^2)$$

The standard deviation σ is chosen based on the sensitivity, ϵ , and δ :

$$\sigma = \frac{\sqrt{2 \ln(1.25/\delta)} \cdot \Delta f}{\epsilon}$$

In our project, these mechanisms are relevant because we compare different ways of adding noise to reduce privacy leakage. The important point is not only that noise is added, but also where it is added and how much is added. Therefore, we test several values of ϵ and compare both model utility and attack success.

2.3 Centralized and Local Differential Privacy

Differential privacy can be used in different settings. In centralized differential privacy, there is a trusted data holder who has access to the original data. The privacy mechanism is applied before publishing a result or a trained model. This can often keep better utility, because the model can still learn from the real data before noise is added.

Local differential privacy uses a stronger privacy setting. Here, the data is randomized before it is collected. This means that the data collector never sees the exact original value. This is useful when the data collector should not be fully trusted. The disadvantage is that the training data is already noisy, so the model usually has a harder time learning useful patterns.

A classical local privacy technique is randomized response, which was introduced by Warner (1965) for sensitive survey questions. The idea is that a person does not always answer truthfully, but follows a random rule. This hides the individual answer, while still allowing useful statistics over many people.

In our project, randomized response is used as a local differential privacy mechanism on the input features. For a feature with k possible values, the true value is kept with probability

$$p = \frac{e^\epsilon}{e^\epsilon + k - 1}$$

Otherwise, the value is replaced by another possible value from the same feature domain. For small ϵ , the true value is replaced more often. This gives stronger privacy, but it also damages the data more. For larger ϵ , the true value is kept more often, so the model can usually learn better.

2.4 Membership Inference Attacks

Membership inference attacks are attacks against machine learning models. The attacker wants to decide whether a specific data point was part of the training set or not. This is especially problematic for medical datasets. For example, if someone can infer that a person was included in a diabetes dataset, then this already reveals sensitive information about that person.

One reason why membership inference attacks can work is that models often behave differently on training data than on unseen data. If a model is much more confident on samples it has already seen during training, then an attacker can use this confidence difference. Shokri et al. (2017) introduced a shadow-model based membership inference attack in a black-box setting. In this attack, the attacker trains additional models that imitate the target model. These shadow models are then used to learn the difference between member and non-member samples based on the model outputs.

A simpler attack is a threshold-based membership inference attack. In this attack, the attacker looks at the model's confidence score for the correct class. If the confidence is above a chosen threshold, the sample is predicted as a member. This attack is simple, but it is still useful because it directly checks whether the model is more confident on training samples than on test samples.

Overfitting is closely related to membership inference. If a model memorizes the training data too strongly, then the difference between training and test behavior can become larger. Yeom et al. (2018) analyze this connection and show that overfitting can increase privacy risk, although it is not the only reason why membership inference attacks work. This is why we also include an intentionally overfitting MLP in our experiments. It acts as a stress test and helps us see whether stronger memorization also leads to stronger privacy leakage.

Overall, these concepts describe the main trade-off in our project. Differential privacy mechanisms try to hide the influence of individual records by adding noise. Membership inference attacks try to detect this influence from the model outputs. In the experiments, we therefore look at both sides: whether the model still performs well, and whether the privacy mechanisms reduce the attacker's success.

3 Dataset and Preprocessing

For the experiments we use the CDC Diabetes Health Indicators dataset Teboul (2015). The dataset is based on survey data from the Behavioral Risk Factor Surveillance System (BRFSS) and contains health-related features such as blood pressure, cholesterol, BMI, smoking, physical activity, age, education, and income. These features are useful for predicting diabetes, but they are also sensitive because they describe personal health and lifestyle information. This makes the dataset suitable for our project, since we want to study both machine learning utility and privacy risks.

The original target column in the dataset is `Diabetes_012`. It contains three possible classes: no diabetes, prediabetes, and diabetes. In our project we simplify the task into a binary classification problem. We remove the prediabetes class and only keep records for people with no diabetes and people with diabetes. After that, the target variable is converted into a binary label:

$$y = \begin{cases} 1, & \text{if the person has diabetes,} \\ 0, & \text{otherwise.} \end{cases}$$

This makes the task easier to evaluate and also matches the original goal of predicting whether a person has diabetes or not.

Before training the models, duplicate rows are removed. This is especially important for membership inference attacks. If the same record appears multiple times, it can make the attack setup less clean, because a record could appear in both training and test data or give the model repeated exposure to the same information. Removing duplicates therefore gives a more realistic and cleaner separation between member and non-member samples.

After preprocessing, the feature matrix X is created by removing the target column `Diabetes_012`, while the binary target vector y is created from this column. The dataset is then split into training and test data. We use a test size of 20%, so 80% of the data is used for training and 20% is used for evaluation.

This preprocessing also defines the membership inference setup. The training set contains the member samples, because these are the records used to train the target model. The test set contains non-member samples, because these records were not used during training. The attacks later compare the model behavior on both groups. If the model is much more confident on training samples than on test samples, this can indicate privacy leakage.

Overall, the preprocessing keeps the project focused on a clear binary health prediction task. At the same time, it prepares the data in a way that is suitable for comparing utility, privacy mechanisms, and membership inference attacks.

4 Methods

This section describes the implemented models, privacy mechanisms, and attacks. The goal is not only to train accurate models, but also to check whether the models leak information about their training data. Therefore, every model is evaluated with normal utility metrics and with membership inference attacks.

4.1 Models

We use three different models in the experiments. The first model is Logistic Regression. This is our main baseline model, because it is simple, stable, and gives probability scores. These probability scores are important for our project, because membership inference attacks often use confidence differences between training and test samples. Logistic Regression is also trained with balanced class weights, because the diabetes dataset is imbalanced and contains more non-diabetic than diabetic samples.

The second model is a standard Multi-Layer Perceptron (MLP). This model is more flexible than Logistic Regression and can learn non-linear relations between the health features and therefore gives a useful comparison between a linear model and a neural network model.

The third model is an intentionally overfitting MLP. This model has a larger architecture and is trained without early stopping. It is also trained on a smaller subset of the training data. The reason for this model is that membership inference attacks are often stronger when a model overfits. Therefore, the overfitting MLP acts as a stress test. If a privacy mechanism can reduce leakage for this model, then this is more interesting than only testing very stable models where the attack is already close to random guessing.

4.2 Privacy Mechanisms

We compare four different training settings. The first one is the non-private baseline. In this setting, the model is trained normally without any privacy protection. This gives us a reference point for both utility and privacy leakage.

The second mechanism is Laplace parameter noise. Here, the model is first trained normally and then random noise from a Laplace distribution is added to the learned model parameters. In our implementation, this is used as an empirical noise baseline. We do not treat it as a fully formal DP guarantee, because the implementation does not include a complete proof of sensitivity for all model types. Still, it is useful for seeing how parameter noise affects utility and membership inference risk.

The third mechanism is Gaussian DP Logistic Regression. This mechanism is only implemented for Logistic Regression. Before training, public feature bounds are used to scale the data, and each row is normalized to keep the input norm bounded. Then a regularized logistic regression model is trained, and Gaussian noise is added to the learned parameters. The noise depends on the privacy budget ϵ , the value of δ , the number of training samples, the row norm bound, and the regularization strength. This is the most principled centralized DP mechanism in our implementation.

The fourth mechanism is local differential privacy using randomized response. In this setting, the input data is randomized before the model is trained. For each feature value, the true value is kept with probability

$$p = \frac{e^\epsilon}{e^\epsilon + k - 1},$$

where k is the number of possible values in the feature domain. Otherwise, the value is replaced by another value from the same domain. This means that the model is trained on already privatized data. The advantage is that the data collector does not directly see the original feature values. The disadvantage is that the model has to learn from noisier data, which can reduce utility.

4.3 Membership Inference Attacks

We evaluate privacy leakage with two membership inference attacks. The first attack is a threshold-based attack. It uses the model’s confidence in the correct class. If this confidence is above a chosen threshold, the sample is predicted as a member of the training set. The threshold is selected so that the attack accuracy is maximized on a balanced set of member and non-member samples.

The second attack is a shadow-model attack, based on the idea of training auxiliary models to imitate the target model (Shokri et al., 2017). In our implementation, several shadow models are trained on

auxiliary data. Their training samples are labeled as members, and their held-out samples are labeled as non-members. From the shadow models, we extract attack features such as predicted probabilities, true-class confidence, maximum confidence, entropy, loss, correctness, and the true label. A Random Forest classifier is then trained as the attack model and evaluated against the target model.

This gives us two different views of privacy leakage. The threshold attack is simple and easy to interpret, while the shadow attack is stronger and more realistic. In the results, an attack AUC close to 0.5 means that the attack is close to random guessing. Values clearly above 0.5 indicate that the model leaks membership information.

5 Experimental Setup

The experiments compare model utility and privacy leakage for different privacy mechanisms and privacy budgets. For all privacy mechanisms except the non-private baseline, we test $\epsilon \in \{0.1, 0.5, 1.0, 5.0\}$. For the Gaussian mechanism, we use $\delta = 10^{-5}$.

We evaluate three models: Logistic Regression, MLP, and overfitting MLP. The privacy mechanisms are the non-private baseline, Laplace parameter noise, Gaussian DP, and local differential privacy with randomized response. Gaussian DP is only evaluated for Logistic Regression.

The experiments are repeated for five trials. For each run, we train the model, evaluate the model on the test set, run the two membership inference attacks, and record the runtime. The final results are reported as mean and standard deviation over the trials.

Component	Values
Dataset	CDC Diabetes Health Indicators
Task	Binary diabetes classification
Models	Logistic Regression, MLP, overfitting MLP
Privacy mechanisms	None, Laplace, Gaussian, LDP randomized response
Privacy budgets	$\epsilon \in \{0.1, 0.5, 1.0, 5.0\}$
Gaussian parameter	$\delta = 10^{-5}$
Trials	5
Utility metrics	Accuracy, F1-score, AUC
Attack metrics	Threshold MIA accuracy/AUC, Shadow MIA accuracy/AUC
Runtime metrics	Training runtime, attack runtime, total runtime

Table 1: Overview of the experimental setup.

The utility metrics are accuracy, F1-score, and AUC. Accuracy gives the overall fraction of correct predictions, but it can be misleading for imbalanced datasets. Therefore, F1-score and AUC are especially important in our setting. The attack metrics are attack accuracy and attack AUC for both the threshold-based and the shadow-model attack. For privacy, attack AUC is especially useful, because it shows how well the attack separates members from non-members. A value close to 0.5 means that the attack is not much better than guessing.

6 Results

This section presents the main experimental results. The focus is on the figures and tables that show the privacy–utility trade-off most clearly. In the plots, the non-private baseline is used as the reference point. For the private mechanisms, the x-axis shows the privacy budget ϵ .

6.1 Model Utility

For Logistic Regression, the non-private baseline reaches an AUC of 0.8141 and an F1-score of 0.4640. This is the main reference point for the utility results. Figure 1 shows that Laplace parameter noise stays almost identical to the non-private baseline for all tested values of ϵ . This means that, for Logistic Regression, the Laplace noise used in our implementation does not visibly damage the ranking quality of the model. The Gaussian mechanism behaves differently. Its AUC is lower, around 0.773 to 0.774, and it does not recover much when ϵ is increased. Local DP with randomized response shows the clearest ϵ trend: at $\epsilon = 0.1$ the AUC is 0.7672, while at $\epsilon = 5.0$ it increases to 0.8013. This is still below the non-private baseline, but it shows that weaker local privacy keeps more useful information in the input features.

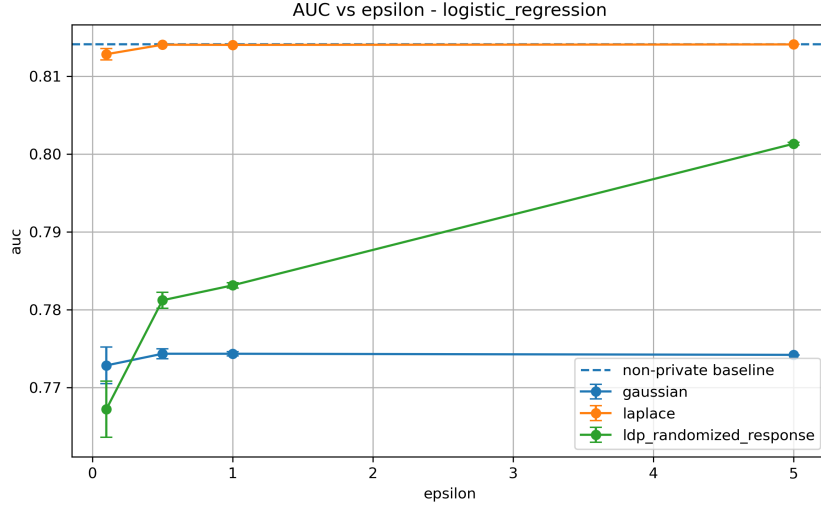


Figure 1: AUC of Logistic Regression for different privacy mechanisms. The dashed line is the non-private baseline. Laplace stays almost on the baseline, Gaussian is lower for all tested privacy budgets, and local DP improves as ϵ increases.

Figure 2 shows the same comparison with F1-score. This metric is important because the diabetes dataset is imbalanced and accuracy alone can hide poor performance on the diabetes class. The F1-score confirms the same general pattern as the AUC plot. Laplace stays close to the baseline, Gaussian has a lower F1-score of about 0.415 to 0.423, and local DP improves from 0.4222 at $\epsilon = 0.1$ to 0.4507 at $\epsilon = 5.0$. Therefore, for Logistic Regression, local DP becomes more useful when the privacy budget is larger, but it still does not fully reach the non-private model.

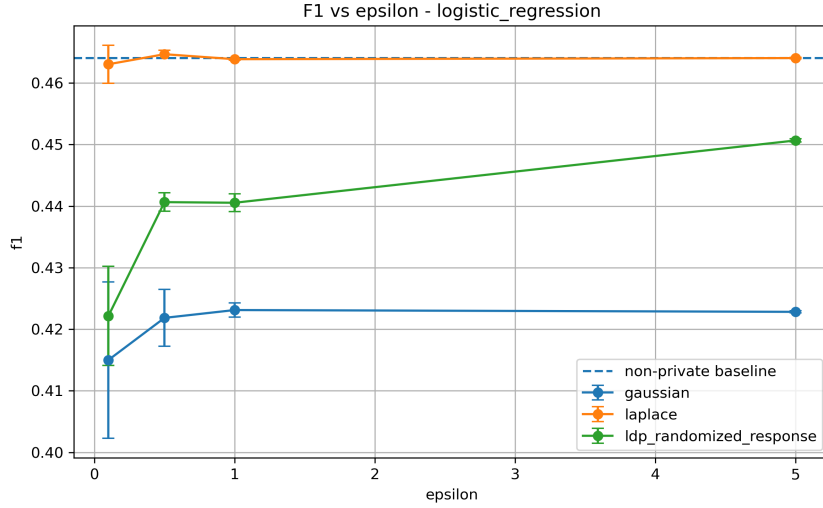


Figure 2: F1-score of Logistic Regression for different privacy mechanisms. The plot shows that the same privacy mechanisms that reduce AUC also reduce the balance between precision and recall for the positive diabetes class.

For the standard MLP, the non-private model reaches an AUC of 0.8210 and an F1-score of 0.4629. Its AUC is slightly higher than Logistic Regression, while the F1-score is almost the same. Therefore, the standard MLP ranks samples a little better, but it does not clearly improve the final binary classification quality. Laplace noise again stays close to the non-private MLP. Local DP has a much stronger effect. At $\epsilon = 0.1$, the MLP drops to an AUC of 0.6639 and an F1-score of 0.3443. At $\epsilon = 5.0$, it recovers to an AUC of 0.8140 and an F1-score of 0.4518. This shows that randomized response can strongly damage neural network utility when the privacy setting is strict, but the model becomes usable again when less noise is added.

The overfitting MLP behaves differently from the other two models. Its non-private accuracy is high at 0.8106, but its F1-score is only 0.3220 and its AUC is only 0.7221. This means that the high accuracy is misleading and is partly caused by the class imbalance. The model does not separate the two classes as well as Logistic Regression or the standard MLP. This is important for the later attack results: the overfitting MLP is not the best utility model, but it is the model where membership leakage becomes clearly visible. Strong privacy settings reduce its utility even more. For example, Laplace noise with $\epsilon = 0.1$ reduces the AUC to 0.5161, and local DP with $\epsilon = 0.1$ reduces it to 0.5485.

Model	Mechanism	ϵ	Accuracy	F1-score	AUC
Logistic Regression	None	—	0.7270 ± 0.0000	0.4640 ± 0.0000	0.8141 ± 0.0000
Logistic Regression	Gaussian	0.1	0.6734 ± 0.0521	0.4150 ± 0.0127	0.7729 ± 0.0024
Logistic Regression	LDP	0.1	0.7447 ± 0.0563	0.4222 ± 0.0081	0.7672 ± 0.0036
Logistic Regression	LDP	5.0	0.7190 ± 0.0005	0.4507 ± 0.0003	0.8013 ± 0.0002
MLP	None	—	0.7206 ± 0.0112	0.4629 ± 0.0039	0.8210 ± 0.0005
MLP	LDP	0.1	0.6819 ± 0.0633	0.3443 ± 0.0398	0.6639 ± 0.0458
MLP	LDP	5.0	0.7018 ± 0.0064	0.4518 ± 0.0032	0.8140 ± 0.0034
Overfit MLP	None	—	0.8106 ± 0.0035	0.3220 ± 0.0134	0.7221 ± 0.0116
Overfit MLP	Laplace	0.1	0.7725 ± 0.0323	0.1442 ± 0.0606	0.5161 ± 0.0600
Overfit MLP	LDP	0.1	0.7820 ± 0.1047	0.1049 ± 0.0464	0.5485 ± 0.0876
Overfit MLP	LDP	5.0	0.8086 ± 0.0077	0.3036 ± 0.0126	0.7079 ± 0.0178

Table 2: Selected utility results. Values are reported as mean $\pm$ standard deviation over five trials.

6.2 Membership Inference Attack Performance

For the membership inference results, an attack AUC close to 0.5 means that the attacker is close to random guessing. Values clearly above 0.5 mean that the attacker can distinguish training samples from test samples better than chance.

For Logistic Regression, both attacks stay close to random guessing. The non-private baseline has a threshold MIA AUC of 0.4935 and a shadow MIA AUC of 0.5022. Figure 3 shows that the shadow attack remains close to 0.5 for all mechanisms and all tested privacy budgets. This means that Logistic Regression does not show strong membership leakage in our setup. Because of that, privacy mechanisms cannot reduce the attack much further. Their main visible effect for this model is therefore a change in utility, not a large change in privacy leakage.

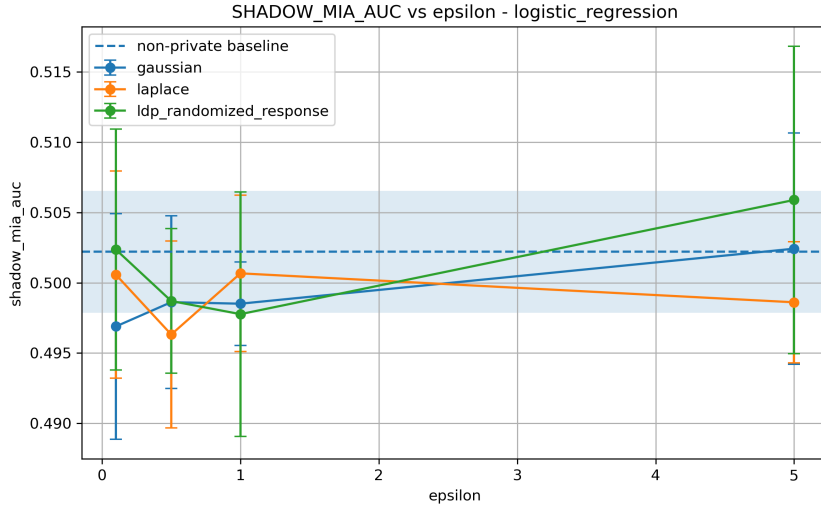


Figure 3: Shadow-model membership inference AUC for Logistic Regression. All values are close to 0.5, so the shadow attack is almost random guessing for this model.

The standard MLP shows the same privacy pattern. Although it is more flexible than Logistic Regression, the attack AUC values also stay around 0.5. The non-private standard MLP has a threshold MIA AUC of 0.4999 and a shadow MIA AUC of 0.4991. This suggests that the regular MLP does not memorize the training data strongly enough for our attacks to exploit it.

The overfitting MLP gives the clearest attack result. The non-private overfitting MLP has a threshold MIA AUC of 0.5741 and a shadow MIA AUC of 0.7036. The shadow attack is therefore much stronger than the threshold attack on this model. This makes sense, because the shadow attack uses several features from the model output, while the threshold attack only uses one confidence score. These results show that the overfitting MLP leaks membership information much more clearly than Logistic Regression and the standard MLP.

Figure 4 shows how the shadow attack changes under privacy mechanisms for the overfitting MLP. With strong noise, the leakage is strongly reduced. Laplace noise with $\epsilon = 0.1$ reduces the shadow MIA AUC from 0.7036 to 0.5044, and local DP with $\epsilon = 0.1$ reduces it to 0.4924. Both values are close to random guessing. When ϵ becomes larger, the attack becomes stronger again. For Laplace noise with $\epsilon = 5.0$, the shadow MIA AUC rises to 0.6907, which is again close to the non-private value. Local DP with $\epsilon = 5.0$ reaches 0.6298, so it still reduces leakage compared to the non-private model, but the attack is clearly better than random guessing.

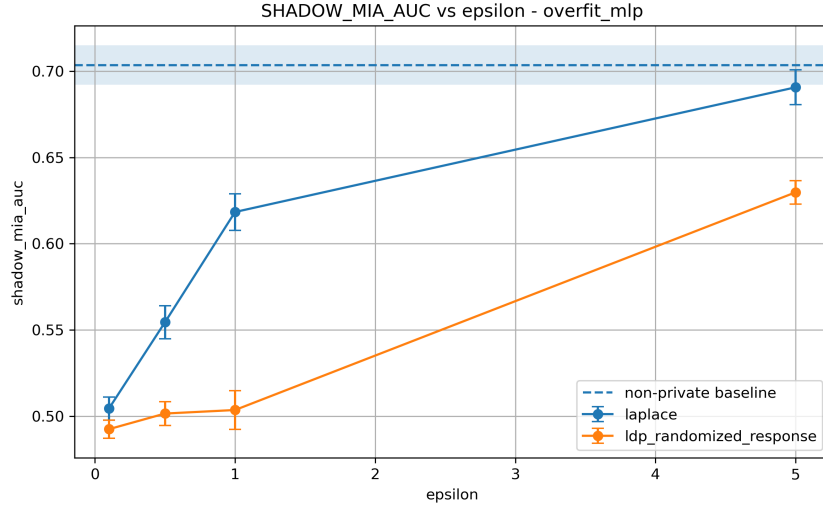


Figure 4: Shadow-model membership inference AUC for the overfitting MLP. Strong privacy settings reduce the attack close to random guessing, while larger ϵ values allow more membership leakage again.

Figure 5 shows the same model with the threshold attack. The trend is similar, but the attack is weaker overall. The non-private threshold MIA AUC is 0.5741, compared to 0.7036 for the shadow attack. This confirms that the shadow attack is the more informative privacy test in our project. Still, the threshold attack is useful because it shows that even a simple confidence-based attack can detect some membership signal in the overfitting model.

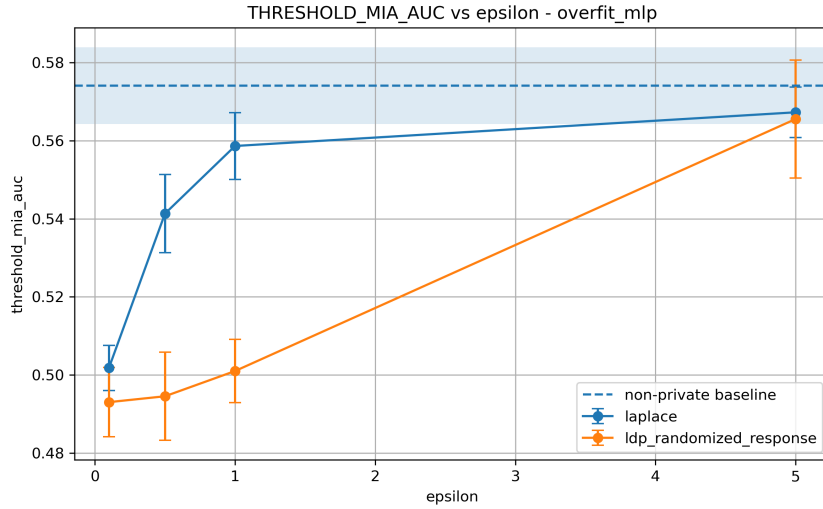


Figure 5: Threshold-based membership inference AUC for the overfitting MLP. The attack is weaker than the shadow attack, but it follows the same general trend: stronger privacy reduces leakage and larger ϵ values increase it again.

Model	Mechanism	ϵ	Threshold MIA AUC	Shadow MIA AUC
Logistic Regression	None	–	0.4935 ± 0.0065	0.5022 ± 0.0043
Logistic Regression	Gaussian	0.1	0.4976 ± 0.0042	0.4969 ± 0.0080
Logistic Regression	LDP	0.1	0.4958 ± 0.0044	0.5024 ± 0.0086
MLP	None	–	0.4999 ± 0.0052	0.4991 ± 0.0067
MLP	LDP	0.1	0.4950 ± 0.0072	0.4970 ± 0.0060
Overfit MLP	None	–	0.5741 ± 0.0098	0.7036 ± 0.0114
Overfit MLP	Laplace	0.1	0.5018 ± 0.0058	0.5044 ± 0.0068
Overfit MLP	Laplace	5.0	0.5673 ± 0.0065	0.6907 ± 0.0100
Overfit MLP	LDP	0.1	0.4930 ± 0.0089	0.4924 ± 0.0053
Overfit MLP	LDP	5.0	0.5655 ± 0.0151	0.6298 ± 0.0068

Table 3: Selected membership inference attack results. Values close to 0.5 mean that the attack is close to random guessing.

6.3 Runtime

The runtime results should be interpreted as implementation measurements and not as formal complexity results. They include model training, attack evaluation, and the overhead of the privacy mechanism. For Logistic Regression, the non-private baseline has a total runtime of 0.90 seconds. Gaussian noise and Laplace parameter noise at $\epsilon = 0.1$ are very close to this, with 0.85 and 0.84 seconds. Local DP at $\epsilon = 0.1$ is much slower for Logistic Regression, with a total runtime of 10.12 seconds. This is visible in Figure 6. In our implementation, randomized response has to process the input features before training, and this preprocessing dominates the runtime for the strict local privacy setting.

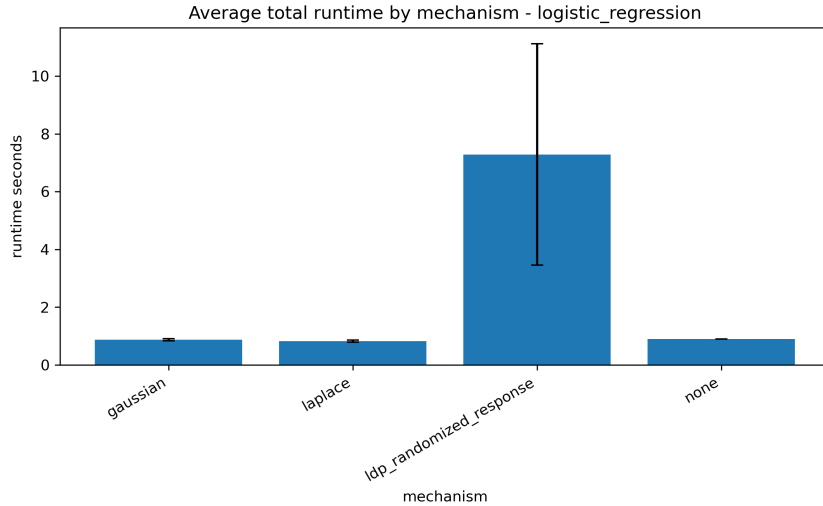


Figure 6: Average total runtime by mechanism for Logistic Regression. Local DP with small ϵ has the largest overhead in this implementation, while the centralized parameter-noise mechanisms are close to the non-private baseline.

For the MLP models, the runtime is generally higher because training neural networks and running the shadow attack takes more time. The standard MLP baseline takes 6.54 seconds, while MLP with local DP at $\epsilon = 0.1$ takes 14.12 seconds. The overfitting MLP baseline takes 10.12 seconds. Interestingly, local DP at $\epsilon = 0.1$ is not slower for the overfitting MLP in the selected results, with a runtime of 8.14 seconds. Therefore, the runtime effect of privacy mechanisms is not completely uniform across all models. It depends on the model, the randomization step, and the cost of the attack evaluation.

Model	Mechanism	ϵ	Total runtime in seconds
Logistic Regression	None	–	0.90 ± 0.18
Logistic Regression	Gaussian	0.1	0.85 ± 0.09
Logistic Regression	Laplace	0.1	0.84 ± 0.12
Logistic Regression	LDP	0.1	10.12 ± 0.41
MLP	None	–	6.54 ± 1.84
MLP	LDP	0.1	14.12 ± 0.66
Overfit MLP	None	–	10.12 ± 1.18
Overfit MLP	LDP	0.1	8.14 ± 0.65

Table 4: Selected runtime results. Values are reported as mean $\pm$ standard deviation over five trials.

7 Discussion

The results show that the privacy–utility trade-off depends strongly on the model. For Logistic Regression, the membership inference attacks are already close to random guessing in the non-private baseline. This means that the model does not show strong membership leakage in our setup. Because of this, the privacy mechanisms cannot reduce the attack much further. Their main effect is on utility. Laplace parameter noise keeps almost the same utility as the baseline, while Gaussian noise and strict local DP reduce AUC and F1-score more clearly.

For the standard MLP, the situation is similar from the privacy side. The model has the highest AUC among our non-private models, but the attack AUC is still close to 0.5. This suggests that the regular MLP does not strongly memorize the training data, at least not in a way that can be exploited by our two attacks. However, local DP with small ϵ clearly damages utility. This is visible in the AUC and F1-score, which both drop strongly at $\epsilon = 0.1$. The model becomes much closer to the baseline again at $\epsilon = 5.0$.

The overfitting MLP gives the clearest privacy result. It has worse AUC and F1-score than the other non-private models, but it leaks much more membership information. The shadow MIA AUC of 0.7036 shows that the attacker can distinguish members from non-members much better than random guessing. This supports the connection between overfitting and membership inference risk. Strong privacy settings reduce this leakage very clearly. Laplace noise and local DP at $\epsilon = 0.1$ both reduce the shadow MIA AUC to around 0.5. However, this comes with a strong utility loss. This is the clearest example of the privacy–utility trade-off in our experiments.

Centralized and local privacy also behave differently. The centralized mechanisms perturb the learned model parameters, so the model can still learn from the original training data before noise is added. This can preserve more utility, especially for Laplace noise in our experiments. Local DP randomizes the input features before training. This gives a stronger trust model, because the original feature values are not directly used by the training algorithm. However, it also makes the learning problem harder. This is visible especially for local DP with $\epsilon = 0.1$, where utility drops strongly for the MLP and the overfitting MLP.

One important limitation is that the Laplace mechanism in our implementation should be interpreted as empirical parameter noise and not as a complete formal DP guarantee. The Gaussian mechanism is more principled, but it is only implemented for Logistic Regression. Another limitation is that the shadow attack uses auxiliary data from the same dataset. This is practical for the project, but it is still a simplified attacker model. Also, five trials are better than a single run, but more repetitions and more datasets would make the results more stable.

Overall, the main finding is not that every privacy mechanism always improves privacy. Instead, the results show that privacy mechanisms are most meaningful when there is actual membership leakage. For Logistic Regression and the standard MLP, the attacks are already close to random guessing. For the overfitting MLP, leakage is clearly visible, and privacy noise can reduce it. However, reducing leakage usually also reduces model utility, especially when the privacy budget is small.

8 Conclusion

In this project, we compared centralized and local privacy mechanisms against membership inference attacks on the CDC Diabetes Health Indicators dataset. We trained Logistic Regression, a standard MLP, and an intentionally overfitting MLP. The models were evaluated with normal utility metrics and with two membership inference attacks: a threshold-based attack and a shadow-model attack.

The results show that Logistic Regression and the standard MLP do not leak much membership information in our setup. Their attack AUC values are close to 0.5, meaning that the attacks are almost random guessing. The overfitting MLP behaves differently. It has a much higher shadow MIA AUC in the non-private setting, which shows that overfitting can increase privacy leakage.

The privacy mechanisms reduce this leakage most clearly for the overfitting MLP. Strong privacy settings, especially with small ϵ , can bring the attack AUC close to random guessing. However, this comes at a cost. Stronger noise can reduce AUC, F1-score, and sometimes make the model less useful for detecting the positive diabetes class. Local DP gives a strong privacy setting because the data is randomized before training, but it can also damage utility more than centralized mechanisms.

Overall, the experiments confirm the main privacy–utility trade-off. Privacy mechanisms can reduce membership inference risk, but they have to be chosen carefully. If the model is already stable and does not leak much, adding strong noise may mostly reduce utility. If the model overfits and leaks membership information, privacy mechanisms become much more useful. Future work could include more datasets, more trials, stronger DP accounting, and additional attack models.

9 Use of generative digital tools

The use of AI tools in this project was declared according to the guidance on AI in teaching in Computer Science at the University of Basel: https://dmi.unibas.ch/fileadmin/user_upload/dmi/Studium/Computer_Science/Diverses/Wegleitungen/Dokumente_2023/Guidance_on_AI_in_teaching_in_Computer_Science.pdf.

Generative digital tools were used as supporting tools during the project. They were used for debugging, code suggestions, assistance with plotting scripts, and improving the wording and flow of self-written report sections. The scientific content, experimental design, implementation decisions, interpretation of the results, and final text remain my own responsibility. All AI-assisted outputs were checked, adapted, and corrected manually.

Tool	Purpose of use	Extent and responsibility
ChatGPT	Debugging support, explanation of errors, suggestions for improving code structure, and support with report wording and flow	Used as an assistant for self-written code and text. Suggestions were reviewed, modified, and integrated manually. No final result was accepted without checking.
GitHub Copilot / Code assistant	Code completion and local code suggestions during implementation	Used for small code suggestions and completion during programming. The final code was reviewed, tested, and adapted manually.
ChatGPT / Code assistant	Assistance with creating and improving plotting scripts	Used to support the structure and readability of plotting code. The scripts were checked and adapted to the project data and desired output.
ChatGPT	Language improvement of self-written report sections	Used to improve clarity, wording, and flow of text written by the author. No new scientific arguments or conclusions were generated by the tool.

Table 5: Use of generative digital tools in the project.

References

- Dwork, C., McSherry, F., Nissim, K., and Smith, A. (2006). Calibrating noise to sensitivity in private data analysis. In Halevi, S. and Rabin, T., editors, *Theory of Cryptography*, pages 265–284, Berlin, Heidelberg. Springer Berlin Heidelberg.
- Shokri, R., Stronati, M., Song, C., and Shmatikov, V. (2017). Membership inference attacks against machine learning models. In *2017 IEEE Symposium on Security and Privacy, SP 2017, San Jose, CA, USA, May 22-26, 2017*, pages 3–18. IEEE Computer Society.
- Teboul, A. (2015). Diabetes health indicators dataset. Kaggle dataset. Accessed: 18 June 2026.
- Warner, S. L. (1965). Randomized response: A survey technique for eliminating evasive answer bias. *Journal of the American Statistical Association*, 60(309):63–69.
- Yeom, S., Giacomelli, I., Fredrikson, M., and Jha, S. (2018). Privacy risk in machine learning: Analyzing the connection to overfitting. In *31st IEEE Computer Security Foundations Symposium, CSF 2018, Oxford, United Kingdom, July 9-12, 2018*, pages 268–282. IEEE Computer Society.